



Contemporary Architecture

Chapter 7

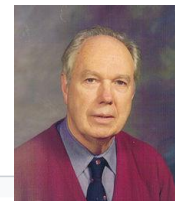


Contemporary Architecture

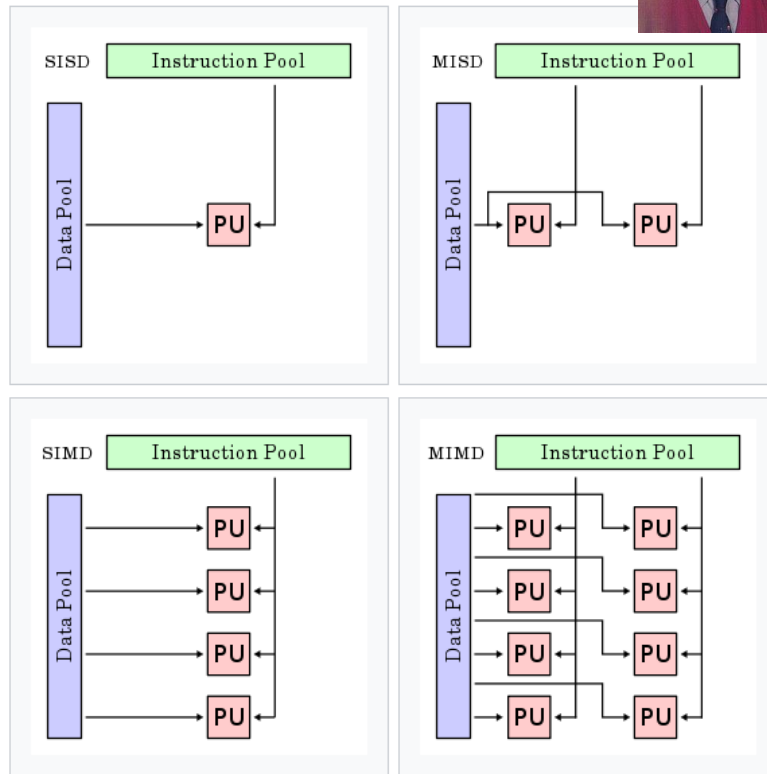
- ★ Flynn's Taxonomy
 - SISD, SIMD, MISD, MIMD
- ★ Instruction-Level Parallelism
 - Vector Processor, Vector Unit
 - Pipelining Processor, SuperPipeline
 - SuperScalar (Out-of-order execution, Register Renaming)
 - Very Long Instruction Word (VLIW)
- ★ SuperSymmetry
- ★ Dynamic vs. Static Scheduling
- ★ Compiler & ILP
 - Loop Unroll, Software Pipelining, LLVM
- ★ Thread-Level Parallelism



Flynn's Taxonomy



- ★ by Michael Flynn (1934-) in 1966
- ★ Partition by Instruction Stream and Data Stream
 - Single Instruction stream, Single Data stream (SISD) - **uniprocessor**
 - Single Instruction stream, Multiple Data stream (SIMD) - **vector processor**
 - Multiple Instruction stream, Single Data stream (MISD) - **Fault-Tolerant Computing**
 - Multiple Instruction stream, Multiple Data stream (MIMD) - **Multicore, Distributed Systems**

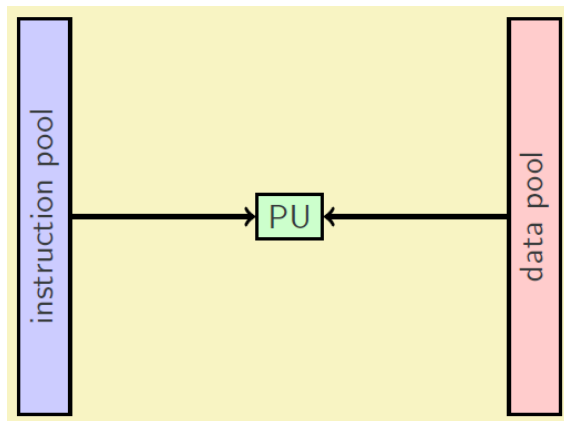


Pictures from wikipedia



Single Instruction stream Single Data stream (SISD)

- A single processing element, executing a single instruction, on a single piece of data, in each step of the execution

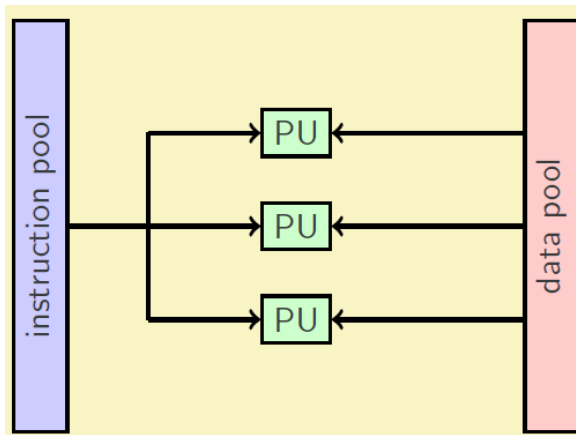


- Standard sequential computer (von Neumann model)
- Pipeline, SuperScalar



Single Instruction stream Multiple Data stream (SIMD)

- Multiple processing elements, executing the same instruction, on multiple pieces of data, in each step of the execution

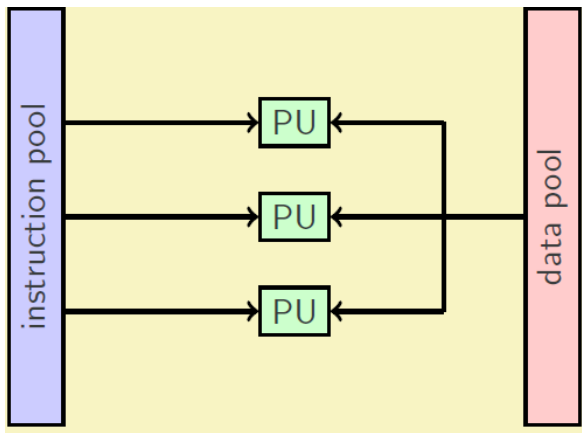


- Graphics Processing Units, Vector Computers
- SSE (Streaming SIMD Extension) registers of modern CPUs



Multiple Instruction stream Single Data stream (MISD)

- Multiple processing elements, executing a separate instruction each, all on the same single piece of data, in each step of the execution

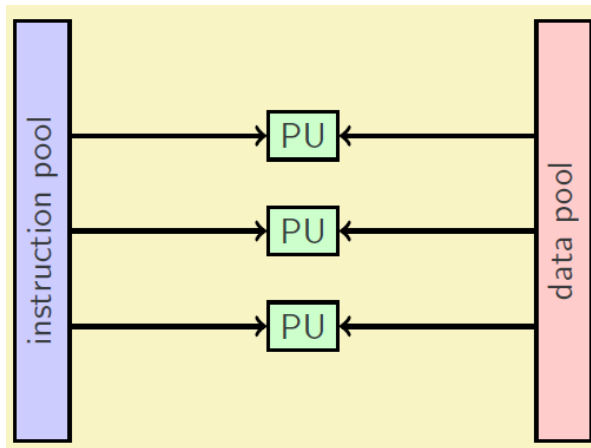


- Fault Tolerant Computing, Dependable Computing
- Security Issues



Multiple Instruction stream Multiple Data stream (MIMD)

- Multiple processing elements, executing different instructions, on a separate piece of data, at each instance of time



- Multicore and multi-processor desktop PCs
- Very Long Instruction Word (VLIW)
- Distributed Computer Systems



Classification of MIMD (Memory Configuration)

- Distributed Memory
 - Every processing element has a certain exclusive portion of the entire memory available in the system. Data needs to be exchanged via an interconnection network.
- Share Memory
 - All processing units in the system can access all data in the main memory.
- Hybrid
 - Certain groups of processing elements share a part of the entire data and instruction storage.



Parallelism

★ Instruction Level Parallelism (ILP)

- ❑ Multiple instructions from one instruction stream are executed simultaneously

★ Thread Level Parallelism

- ❑ Multiple instruction streams are executed simultaneously

★ Data Level Parallelism (DLP)

- ❑ Multiple data are executed simultaneously



Instruction-Level Parallelism

- ★ ILP - a measure of how many of the instructions in a computer program can be executed simultaneously
- ★ Several micro-architectural techniques involve:
 - Vectorization
 - Speculative Execution - allow (parts of) instructions to be executed before needed.
 - Branch Prediction - avoid stalling from control dependencies(speculative execution)
 - Instruction Pipelining and Super Pipeline
 - SuperScalar
 - Out-of-order execution
 - Register Renaming
 - VLIW



Data Level Parallelism

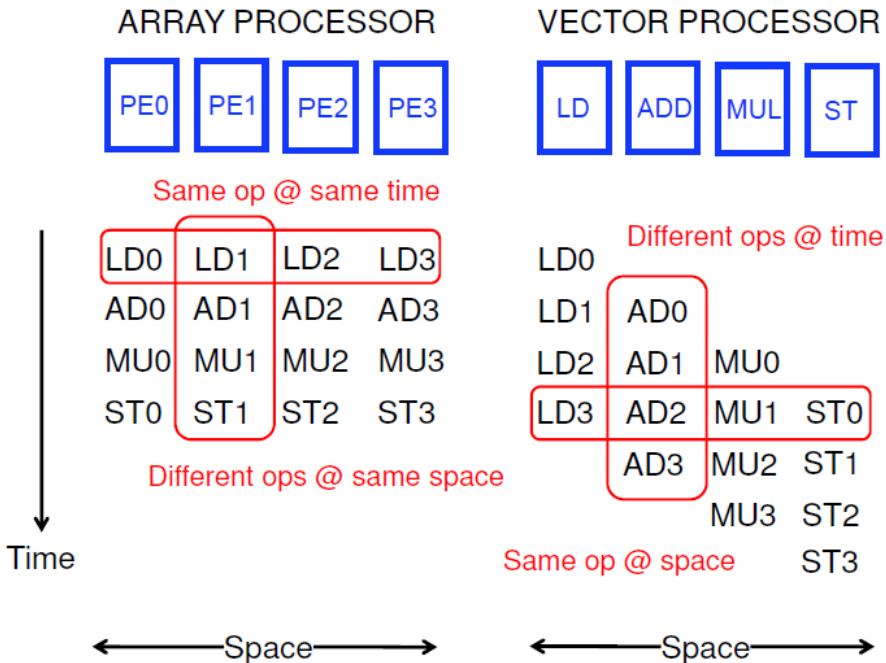
- Concurrency arises from performing the same operations on different pieces of data
- SIMD exploits data-level parallelism
- Time-space duality
 - **Array processor:** Instruction operates on multiple data elements at the same time
 - **Vector processor:** Instruction operates on multiple data elements in consecutive time steps



Array vs. Vector Processors

Instruction Stream

```
LD  VR ← A[3:0]
ADD VR ← VR, 1
MUL VR ← VR, 2
ST  A[3:0] ← VR
```





Vector Processor, Vector Unit

- ★ A variation of SIMD
- ★ Known products
 - Apple/Motorola/IBM AltiVec
 - Intel MMX, SSE[2,3,4], AVX[2,512]
 - GPU - CUDA, OpenCL
- ★ Apply single operation to vectors (array of data)
- ★ We will revisit this on GPGPU topics.

```
// uniprocessor
int A[8];
int B[8];
int C[8];

for (int i=0;i<8;i++)
{
    C[i]=A[i]+B[i];
}
```

```
// vectorize code
// equivalent in C
int A[8];
int B[8];
int C[8];

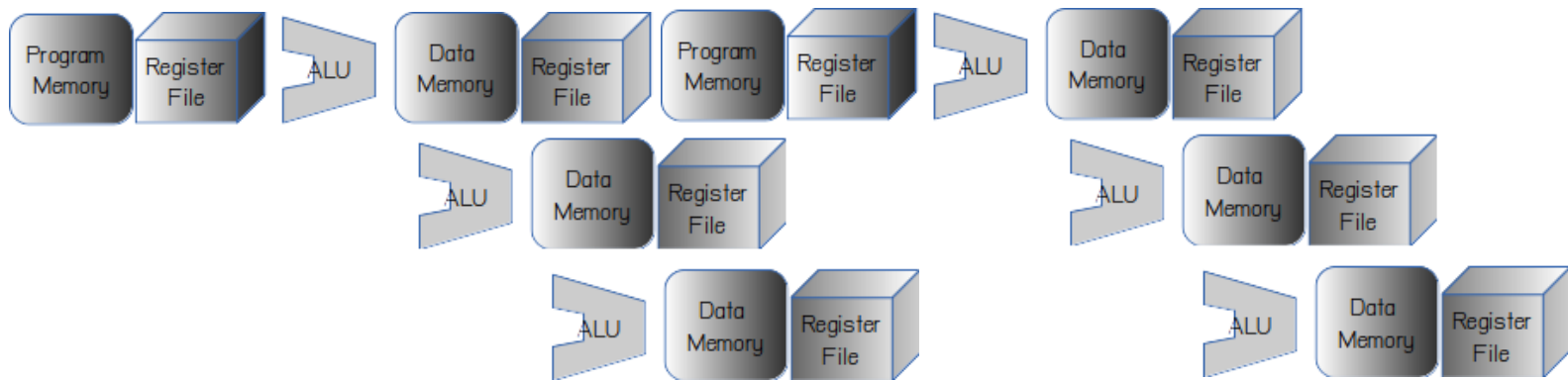
C=A+B;
```

```
; Optimize of -O3 on GCC 6.3
; vectorize with Intel AVX
...
movdqa    16+B(%rip), %xmm0
paddb     16+A(%rip), %xmm0
movaps    %xmm0, 16+C(%rip)
...
```



Vector Processor (ctd.)

★ Some vector processors will expand loops internally.





Vector processor in real world

- ★ Ordering several dishes of the same things, a cook receives an order once.



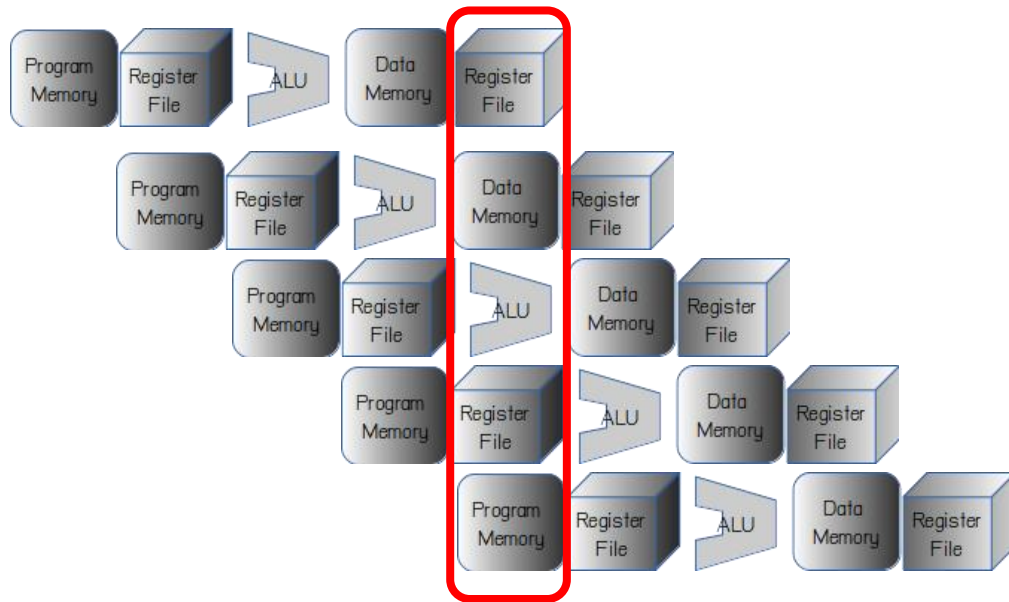
I want 10 cups of noodle.





Pipelining Processor (revisit)

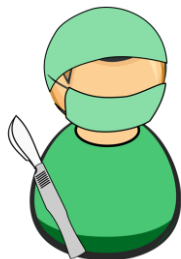
- ★ A pipeline with 5 states has 5 instructions in flight.
- ★ For k state pipelines, $ILP = k$.
- ★ CPU Time for n instructions $T(n) = (k + (n-1)) T_c$





Pipeline in real world

- ★ One line for buying food.
- ★ 5 steps





SuperScalar

- ★ SuperScalar with degree 2 and 5 states of pipeline has 10 instructions in flight.
- ★ For k state pipelines in superscalar x degrees, $ILP = kx$.
- ★ CPU Time for n instructions $T(n) = (k + (n-x)/x) T_c$





SuperScalar in real world

- ★ One line for buying food.
- ★ After getting in, a manager will separate customers into two lines (rice line and noodle line)





SuperScalar (ctd.)

- ★ With super scalar, we may have **out-of-order execution**.
- ★ A person that comes after (but order noodle) may finish earlier than a person that comes before (but order rice).
- ★ This may result in potential **WAR** and **WAW**.
- ★ To solve this -- **in-order completion** ?



Instruction Policies: In-Order / Out-of-Order

- In-Order / Out-of-Order (Issue / Execution / Completion)
- In-order issue, in-order completion

Assume:

- I1 requires 2 cycles to execute;
- I3 vs I4 and I5 vs I6 are in conflict for a functional unit;
- I5 depends on the value produced by I4

Time



Decode		Execute			Writeback	
I1	I2					
I3	I4	I1	I2			
I3	I4	I1				
	I4			I3	I1	I2
I5	I6			I4		
	I6		I5		I3	I4
			I6			
					I5	I6

6 instructions
require 8 cycles

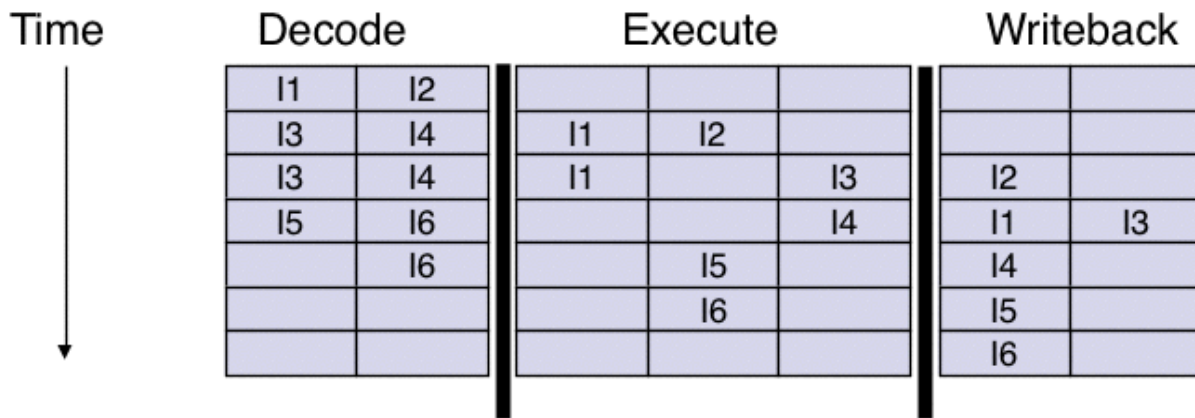
$$IPC = 6/8 = 0.75$$



In-Order Issue, Out-of-Order Completion

Assume:

- I1 requires 2 cycles to execute;
- I3 vs I4 and I5 vs I6 are in conflict for a functional unit;
- I5 depends on the value produced by I4



6 instructions
require 7 cycles

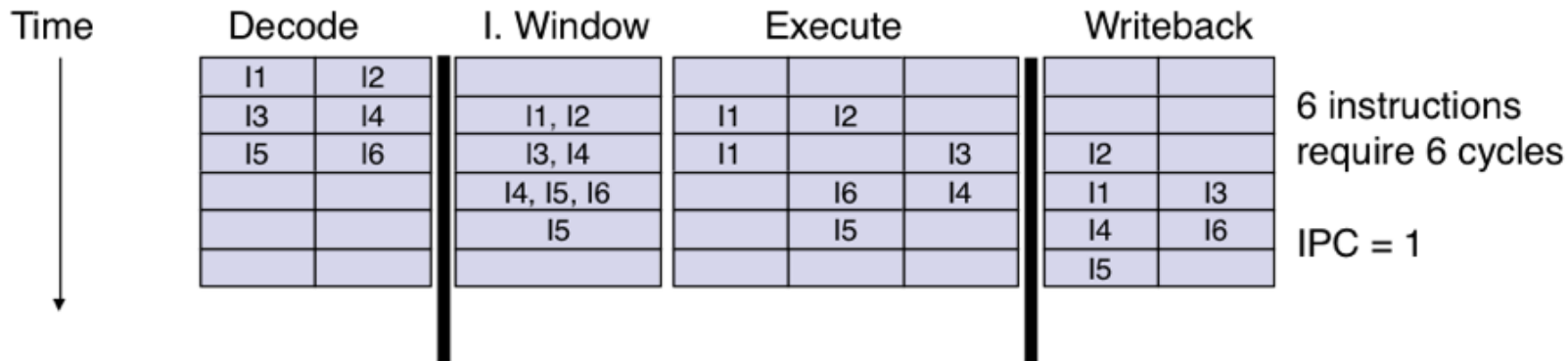
$$IPC = 6/7 = 0.86$$



Out-of-Order Issue, Out-of-Order Completion

Assume:

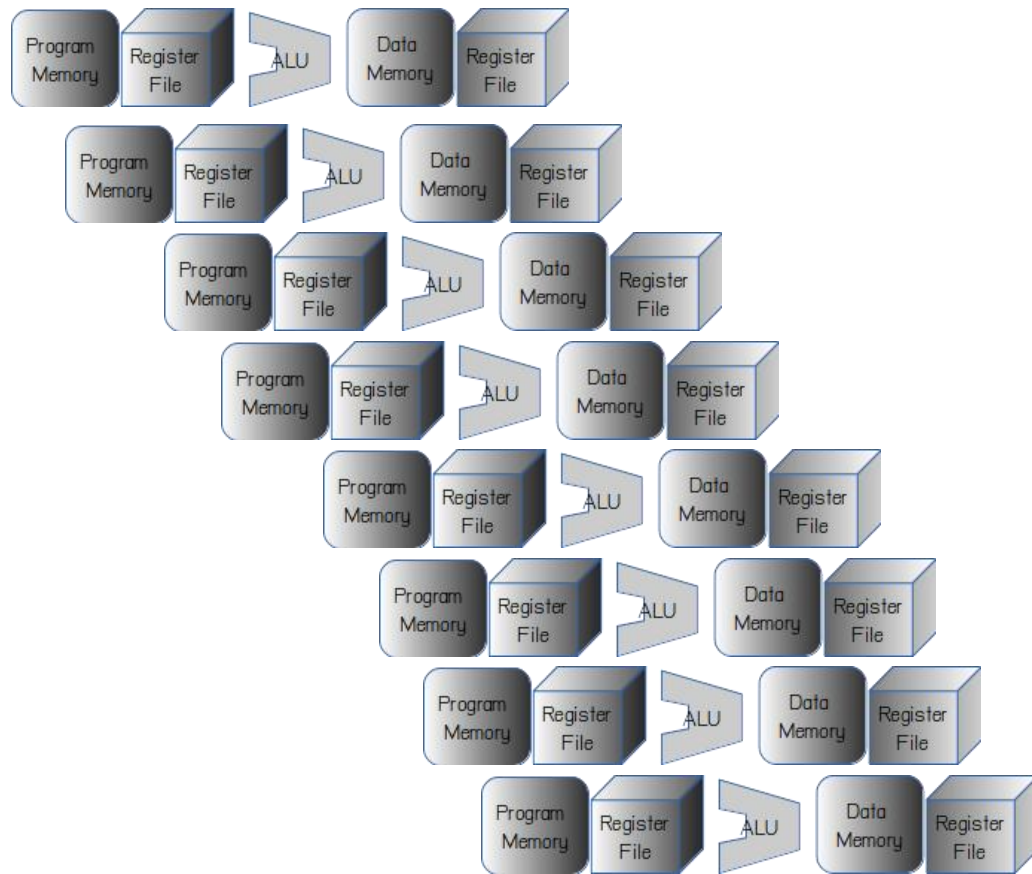
- I1 requires 2 cycles to execute;
- I3 vs I4 and I5 vs I6 are in conflict for a functional unit;
- I5 depends on the value produced by I4
- **Instruction Window** to buffer decoded or part pre-decoded instructions





SuperPipeline

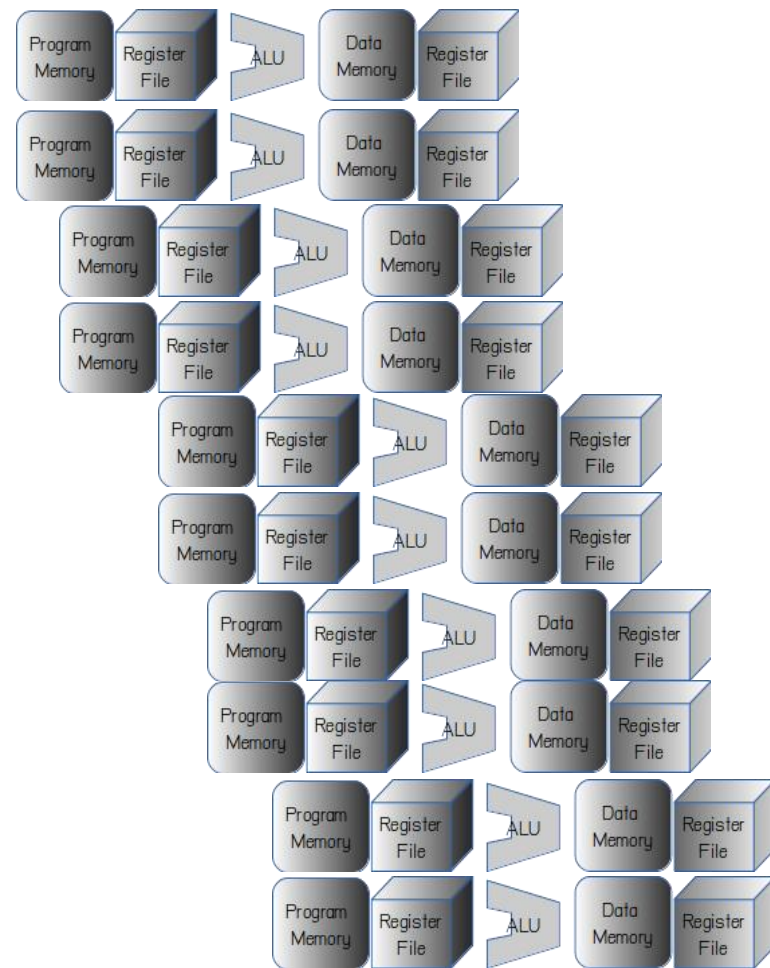
- ★ Use minor cycles.
- ★ In this example, 2 minor cycles is equivalent to 1 major cycle.
- ★ For m minor cycles to k state pipeline, $ILP = mk$
- ★ CPU Time for n instructions $T(n) = (k + (n-1)/m) T_c$





SuperPipeline SuperScalar

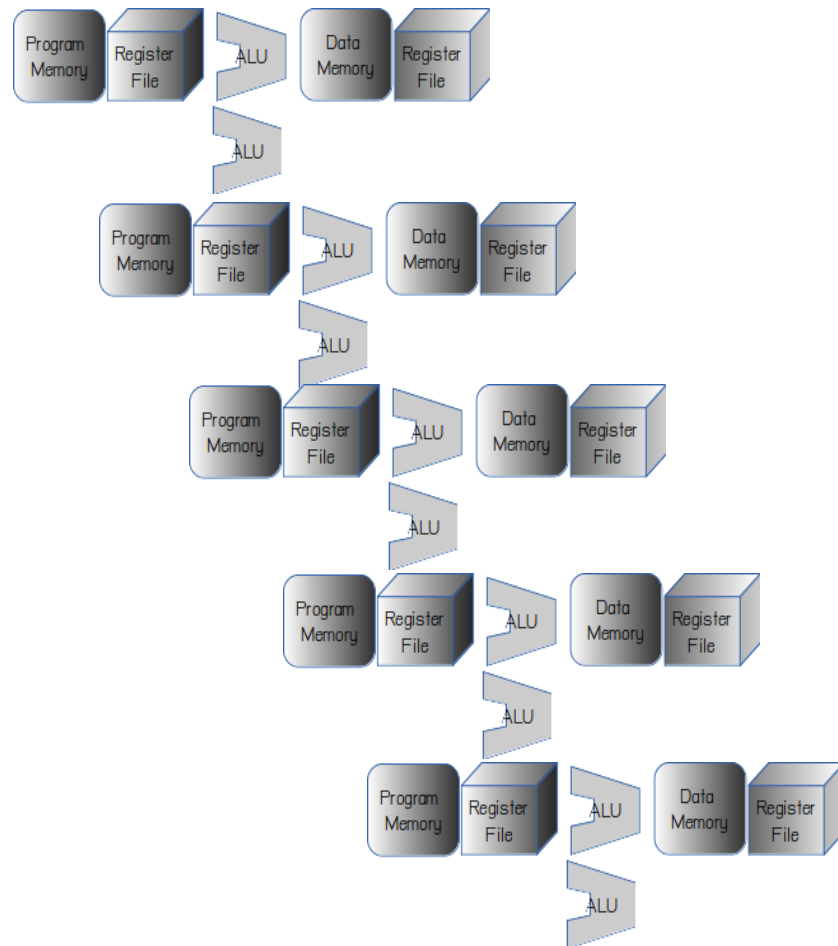
- ★ SuperScalar with minor cycles.
- ★ In this example, 2 minor cycles is equivalent to 1 major cycle.
- ★ For m minor cycles to k states pipeline in superscale degree x,
ILP = mkx
- ★ CPU Time for n instructions
 $T(n) = (k + (n-x)/mx) T_c$





VLIW

- ★ VLIW with degree 2 and 5 states of pipeline has 10 instructions in flight.
- ★ For k state pipelines in VLIW x degrees, $ILP = kx$.
- ★ CPU Time for n major instructions $T(n) = (k + (n/x - x)) T_c$





VLIW in real world



- ★ Static scheduling.
- ★ Two lines for ordering
- ★ One for rice
- ★ Another for noodle



SuperSymmetry

★ SuperScalar

★ Speedup=
$$\frac{(k + (n-x)/x) T_c}{(k + (n-1)) T_c}$$

★ For large n,
Speedup=x

★ SuperPipeline

★ Speedup=
$$\frac{(k + (n-1)/m) T_c}{(k + (n-1)) T_c}$$

★ For large n,
Speedup=m

★ VLIW

★ Speedup=
$$\frac{(k + (n/x-x)) T_c}{(k + (n-1)) T_c}$$

★ For large n,
Speedup=x

At the same degree, SuperScalar, SuperPipeline and VLIW yield the same performance.



Observations on VLIW

- ★ Software for Intel Itanium and Intel Itanium II (also the successors) are not binary compatible. **Why?** (Hint. static scheduling)
- ★ As of January 2019, Intel officially announced the end of life and product discontinuance of the Itanium CPU family.
- ★ Other implementations of VLIW exist (e.g. Transmeta Crusoe). They are not practically used today.



Speed Up (pipeline)

★ Assuming that there are two separate pipelines (integer unit and floating point unit), two instructions can be executed at the sametime.

★ CPU Time = Max (Int, FP)

★ Potential Speed Up

$$= 19 / \text{Max}(\text{Int}, \text{FP})$$

$$= 19 / \text{Max}(7, 12)$$

$$= 1.6$$

Basic Loop:	Cycles
load Ra <- Ai	1
load Ry <- Yi	1
fmult Rm <- Ra * Rx	7
fadd Rs <- Rm + Ry	5
store Ai <- Rs	1
inc Yi	1
dec i	1
inc Ai	1
branch	1



Performance (Revisit)

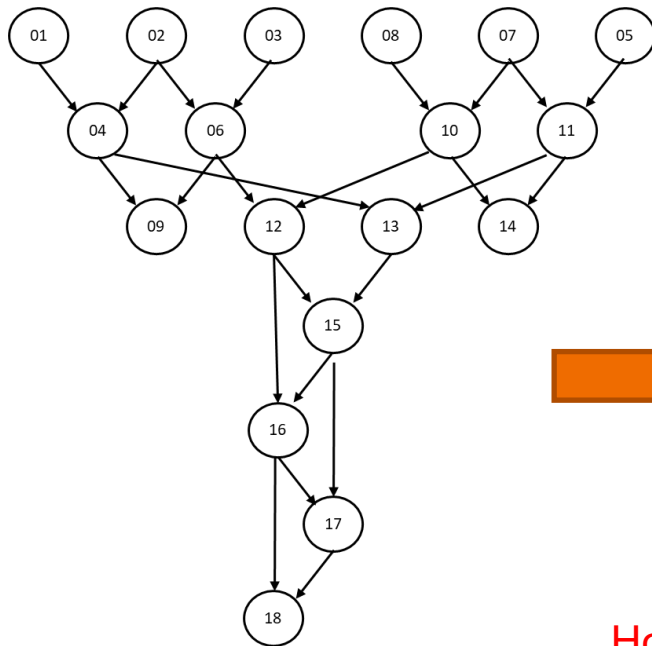
- ★ CPU Time = $IC \times CPI \times TC$
- ★ In practical, operation classes and latency are not balance.

	IC	CPI	TC	Potential Speed Up
Single Cycle Processor	n	1	L	1
Multiple Cycle Processor	n	3-5 (avg=4.2)	L/5	1.xx
Pipelining Processor	n	1	L/5	5
SuperScalar (degree k)	n	1/k	L/5	5k
SuperPipeline (m minor cycle)	n	1/m	L/5	5m
VLIW (x instructions in a word)	n/x	1	L/5	5x



Check yourself: VLIW

01	lw \$t0 ← 0(\$a0)
02	lw \$t1 ← 4(\$a0)
03	lw \$t2 ← 8(\$a0)
04	add \$t6 ← \$t0, \$t1
05	lw \$t3 ← 12(\$a0)
06	sub \$t7 ← \$t1, \$t2
07	lw \$t4 ← 16(\$a0)
08	lw \$t5 ← 20(\$a0)
09	srlv \$s2 ← \$t6, \$t7
10	sub \$s1 ← \$t4, \$t5
11	add \$s0 ← \$t3, \$t4
12	sliv \$s4 ← \$t7, \$s1
13	srlv \$s3 ← \$t6, \$s0
14	sliv \$s5 ← \$s0, \$s1
15	add \$s6 ← \$s3, \$s4
16	add \$s7 ← \$s4, \$s6
17	srlv \$t0 ← \$s6, \$s7
18	srlv \$t1 ← \$t0, \$s7

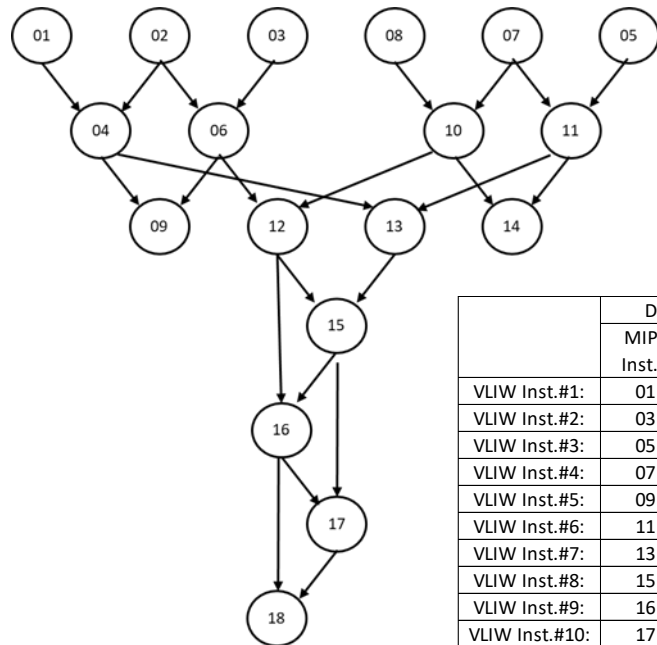


	Degree = 2	
	MIPS Inst.#	MIPS Inst.#
VLIW Inst.#1:	01	02
VLIW Inst.#2:	03	04
VLIW Inst.#3:	05	06
VLIW Inst.#4:	07	08
VLIW Inst.#5:	09	10
VLIW Inst.#6:	11	12
VLIW Inst.#7:	13	14
VLIW Inst.#8:	15	
VLIW Inst.#9:	16	
VLIW Inst.#10:	17	
VLIW Inst.#11:	18	

How about it if the degree = 3 or 6?



Check yourself: VLIW (Solutions)



	Degree = 2		Degree = 3				Degree = 6							
	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#	MIPS Inst.#
VLIW Inst.#1:	01	02	01	02	03		01	02	03	05	07	08		
VLIW Inst.#2:	03	04	04	05	06		04	06	10	11				
VLIW Inst.#3:	05	06	07	08	09		09	12	13	14				
VLIW Inst.#4:	07	08	10	11			15							
VLIW Inst.#5:	09	10	12	13	14		16							
VLIW Inst.#6:	11	12	15				17							
VLIW Inst.#7:	13	14	16				18							
VLIW Inst.#8:	15		17											
VLIW Inst.#9:	16		18											
VLIW Inst.#10:	17													
VLIW Inst.#11:	18													



Contemporary Architecture

- ★ Flynn's Taxonomy
 - SISD, SIMD, MISD, MIMD
- ★ Instruction-Level Parallelism
 - Vector Processor, Vector Unit
 - Pipelining Processor, SuperPipeline
 - SuperScalar (Out-of-order execution, Register Renaming)
 - Very Long Instruction Word (VLIW)
- ★ SuperSymmetry
- ★ Dynamic vs. Static Scheduling
- ★ Compiler & ILP
 - Loop Unroll, Software Pipelining, LLVM
- ★ Thread-Level Parallelism



Compiler & ILP

- ★ Compiler can play a critical role in exploding more ILP
- ★ Small basic block means less opportunity for filling available slots.
- ★ A larger basic block means more opportunities for filling slots.
- ★ To increase a basic block,
 - Loop Unrolling
 - Software Pipeline

- ★ LLVM: a set of compiler and toolchain technologies, which can be used to develop a front-end for any programming language and a back-end for any instruction set architecture.



Improving (Pipeline) Instruction Scheduling

- *Goal:* keep the pipeline full (Avoiding stalls due to hazards)
- *Approach:* Compile-Time Pipeline Scheduling
 - Compiler sequences instructions to minimize stalls
 - Hardware executes instructions in their new order
- *Techniques:*
 - Deal with Data Hazards:
Register renaming, Scheduling loads
 - Deal with Control Hazards:
Loop unrolling, Software pipelining,
Prediction and speculation



Register renaming

- **Idea**

- Reduce data hazards by reducing register reuse
- Give the instruction scheduler greater freedom

- **Example**

add	\$r1, \$r2, 1		add	\$r1, \$r2, 1		add	\$r1, \$r2, 1
st	\$r1, [\$fp+52]		st	\$r1, [\$fp+52]		mul	\$r11, \$r3, 2
mul	\$r1, \$r3, 2	➡	mul	\$r11, \$r3, 2	➡	st	\$r1, [\$fp+52]
st	\$r1, [\$fp+40]		st	\$r11, [\$fp+40]		st	\$r11, [\$fp+40]



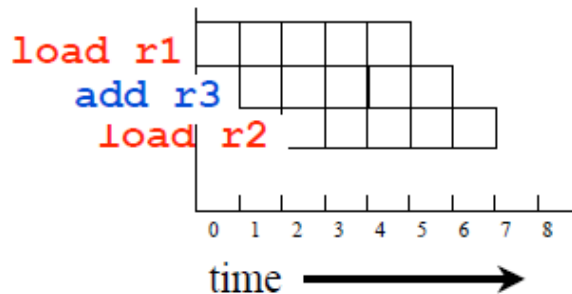
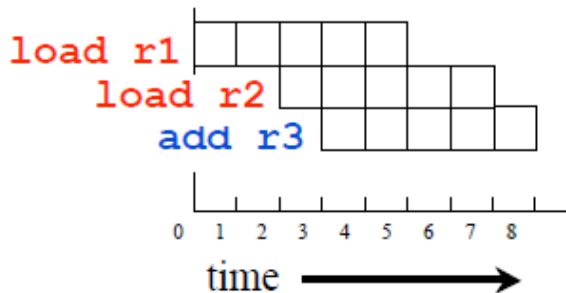
Scheduling loads

- **Reality**

- Loads can take many cycles (slow caches, cache misses)
- Many cycles may be wasted

- **Hiding latency**

- Place independent instructions behind loads





Loop Unroll

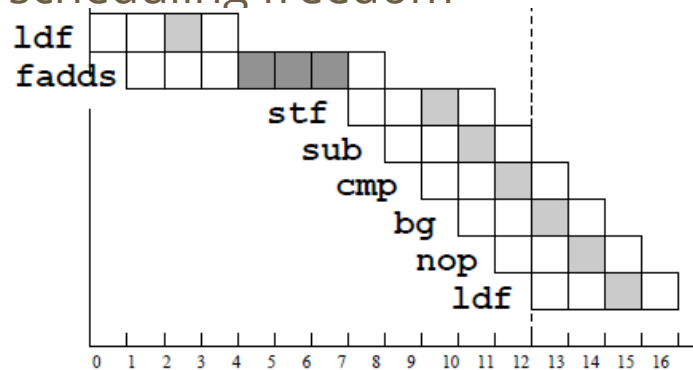
● Idea

- Replicate body of loop and iterate fewer times
- Reduce loop overhead (test and branch)
- Creates larger loop body $\Rightarrow$ more scheduling freedom

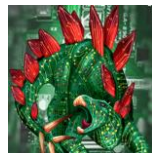
● Example

```
L: ldf    [r1], f0
    fadds f0, f1, f2
    stf    f2, [r1]
    sub    r1, 4, r1
    {
      sub    r1, 4, r1
      cmp    r1, 0
      bg     L
      nop
    }
```

Loop overhead



Cycles per iteration: 12

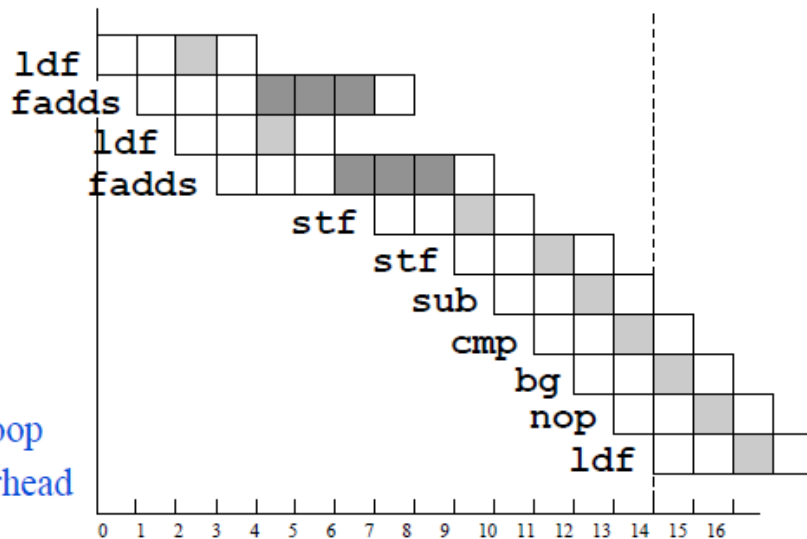


Loop Unroll example

Sample loop

```
L: ldf    [r1], f0
    fadds f0, f1, f2
    ldf    [r1-4], f10
    fadds f10, f1, f12
    stf    f2, [r1]
    stf    f12, [r1-4]
    sub    r1, 8, r1
    cmp    r1, 0
    bg     L
    nop
```

Loop overhead



Cycles per iteration: $14/2 = 7$
(71% speedup!)

The larger window lets us hide some of the latency of the **fadds** instruction



Loop Unroll

- ★ Make the basic block in a loop larger by having more iterations in a loop.
- ★ The new code is potentially not stall at the cost of larger code size.

Normal loop.

```
for (i=MAX-1;i>=0;i--) {  
    A[i] = A[i] + C;  
}
```

Symbolically unroll 4 iterations

```
for (i=MAX-1;i>=3;i-=4) {  
    A[i] = A[i] + C;  
    A[i-1] = A[i-1] + C;  
    A[i-2] = A[i-2] + C;  
    A[i-3] = A[i-3] + C;  
}
```



Loop Unroll (ctd.)

Unroll (4 iterations)

Normal Loop

```
LOOP:
    LW    $f0, $r1
    (stall)
    ADD    $f2, $f0, $f1
    (stall)
    (stall)
    SW    $f2, $r1
    SUB    $r1, $r1, #4
    BNE    $r1, LOOP
```

8 cycles / Iteration

Register Renaming

- ★ To free unrolled iterations from the first iteration, a register would be renamed.
- ★ Each iteration is now resource independent.

```
LOOP:
    LW    $f0, 0($r1)
    (stall)
    ADD    $f2, $f0, $f1
    (stall)
    (stall)
    SW    $f2, 0($r1)
    LW    $f3, 4($r1)
    (stall)
    ADD    $f4, $f3, $f1
    (stall)
    (stall)
    SW    $f4, 4($r1)
    LW    $f5, 8($r1)
    (stall)
    ADD    $f6, $f5, $f1
    (stall)
    (stall)
    SW    $f6, 8($r1)
    LW    $f7, 12($r1)
    (stall)
    ADD    $f8, $f6, $f1
    (stall)
    (stall)
    SW    $f8, 12($r1)
    SUB    $r1, $r1, #16
    BNE    $r1, LOOP
```

Unroll + Rescheduling

```
LOOP:
    LW    $f0, 0($r1)
    LW    $f3, 4($r1)
    LW    $f5, 8($r1)
    LW    $f7, 12($r1)
    ADD    $f2, $f0, $f1
    ADD    $f4, $f3, $f1
    ADD    $f6, $f5, $f1
    ADD    $f8, $f6, $f1
    SW    $f2, 0($r1)
    SW    $f4, 4($r1)
    SW    $f6, 8($r1)
    SW    $f8, 12($r1)
    SUB    $r1, $r1, #16
    BNE    $r1, LOOP
```

14 cycles / 4 Iteration
= 3.5 cycles per iteration

Speed Up=8/3.5



Loop Unroll and Superscalar

12 cycles / 5 Iteration
= 2.4 cycles per iteration

Integer Unit	Floating Unit	Clock Cycle (Issued)
LOOP:		
LW \$f0 , 0(\$r1)		1
LW \$f3, 4(\$r1)		2
LW \$f5, 8(\$r1)	ADD <u>\$f2</u> , \$f0 , \$f1	3
LW \$f7, 12(\$r1)	ADD \$f4, \$f3, \$f1	4
LW \$f9, 16(\$r1)	ADD \$f6, \$f5, \$f1	5
SW <u>\$f2</u> , 0(\$r1)	ADD \$f8, \$f6, \$f1	6
SW \$f4, 4(\$r1)	ADD \$f10, \$9, \$f1	7
SW \$f6, 8(\$r1)		8
SW \$f8, 12(\$r1)		9
SW \$f10, 16(\$r1)		10
SUB \$r1, \$r1, #24		11
BNE \$r1, LOOP		12



Loop Unrolling isn't free

- Benefits decrease with additional unrolling
- Code length increases with additional unrolling
 - This is issue for embedded processors
 - This can increase instruction cache miss rates
- Uses lots of registers
 - Renaming requires many registers
 - When registers become scarce: “register pressure”
 - Aggressive unrolling and scheduling and cause a compiler to run out of registers to use for renaming



Loop Unroll summary

- **Benefit**

- Loop unrolling allows us to schedule code across iteration boundaries, providing more scheduling freedom

- **Issues**

- How much unrolling should we do?
 - Try various unrolling factors and see which provides the best schedule?
 - Unroll as much as possible within a code expansion budget?
- An alternative: **Software pipelining**



Software Pipelining

- **Basic idea**

- **Software pipelining** is a systematic approach to scheduling across iteration boundaries without doing loop unrolling
- Try to move the long latency instructions to **previous** iterations of the loop
- Use independent instructions to hide their latency

- **Three Parts of a Software Pipelining**

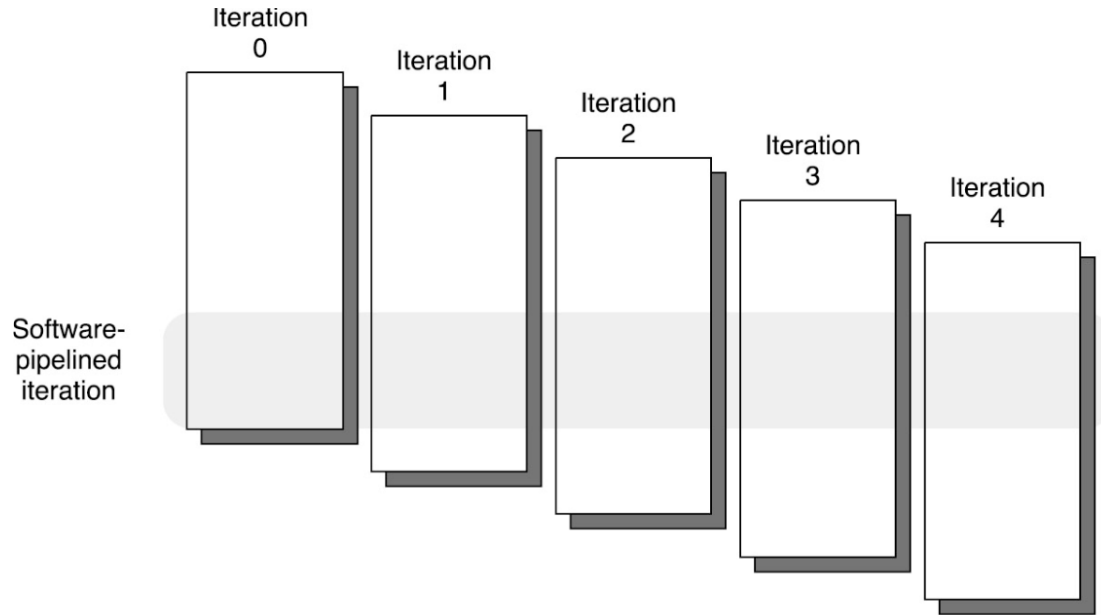
Start up: Code to fill the pipeline

Kernel: Steady state execution of the pipeline

Clean up: Code to empty the pipeline



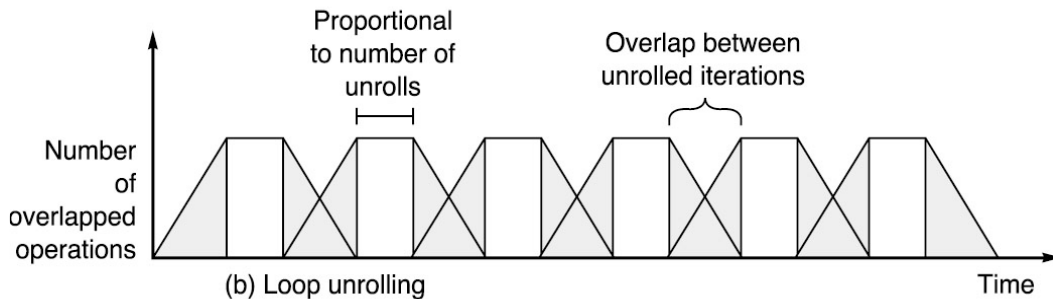
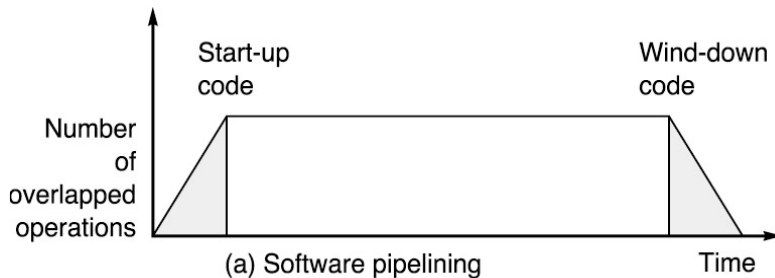
Visualizing Software Pipelining



© 2003 Elsevier Science (USA). All rights reserved.



Software Pipelining vs Loop Unroll



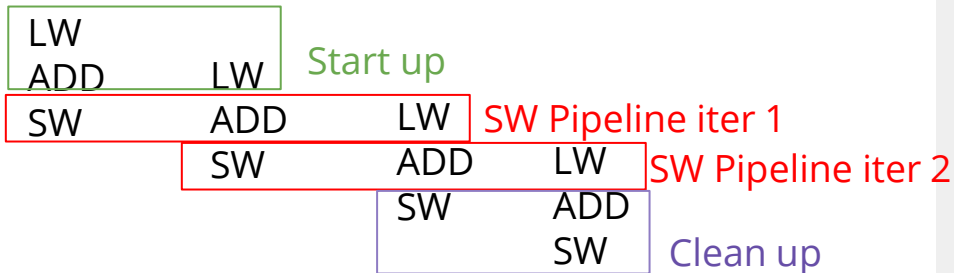
© 2003 Elsevier Science (USA). All rights reserved.



Software Pipeline

- ★ Unroll as needed only.
- ★ Good performance, appropriate code size.

Iter 1 Iter 2 Iter 3 Iter 4



Unroll (3 iterations)

```
LOOP:
    LW    $f0, 0($r1)
    LW    $f3, 4($r1)
    LW    $f5, 8($r1)
    ADD    $f2, $f0, $f1
    ADD    $f4, $f3, $f1
    ADD    $f6, $f5, $f1
    SW     $f2, 0($r1)
    SW     $f4, 4($r1)
    SW     $f6, 8($r1)
    SUB    $r1, $r1, #12
    BNEZ   $r1, LOOP
```

Start up

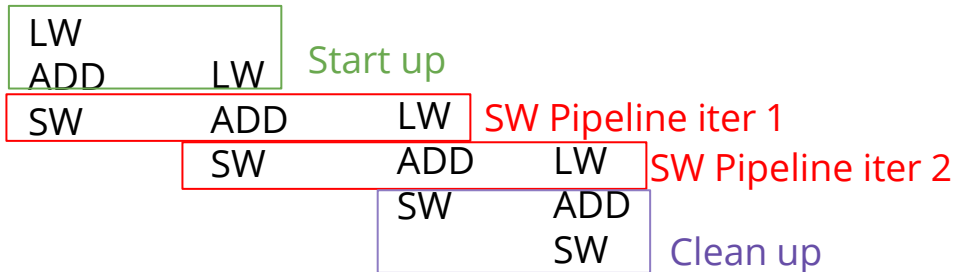
Clean up

Unroll + Rescheduling

```
    SUB    $r1, $r1, #8
    LW     $f0, 8($r1)
    ADD    $f2, $f0, $f1
    LW     $f0, 4($r1)
LOOP:
    SW     $f2, 8($r1)
    ADD    $f2, $f0, $f1
    LW     $f0, 0($r1)
    SUB    $r1, $r1, #4
    BNEZ   $r1, LOOP
    SW     $f2, 4($r1)
    ADD    $f2, $f0, $f1
    SW     $f2, 0($r1)
```



Iter 3 Iter 2 Iter 1 Iter 0



```
int C = 8;
int A[4];
for(i = 3; i >= 0; i--)
    A[i] = A[i] + C;
```

Start up

```
SUB    $r1, $r1, #8
LW     $f0, 8($r1)
ADD    $f2, $f0, $f1
LW     $f0, 4($r1)
LOOP:
SW     $f2, 8($r1)
ADD    $f2, $f0, $f1
LW     $f0, 0($r1)
SUB    $r1, $r1, #4
BNEZ   $r1, LOOP
```

Clean up

```
SW     $f2, 4($r1)
ADD    $f2, $f0, $f1
SW     $f2, 0($r1)
```

```
i = 1
f0 <- A[3]
f2 <- f0+C
f0 <- A[2]
Loop:r1 holds index m
    A[m+2] <- f2
    f2 <- f0+C
    f0 <- A[m]
    m <- m-1
    Stop if m==0

A[1] <- f2
f2 <- f0 + C    #f0 holds A[0]
A[0] <- f2
```



Dynamic vs. Static Scheduling

★ Dynamic Scheduling (Hardware)

- ★ Hardware dynamically schedules code (out-of-order execution).
 - Loop are dynamically unrolled in the processor.
 - Multiple instructions are dynamically issued.
- ★ Legacy software gains better performance on new hardware.
- ★ Complex hardware design?

★ Static Scheduling (Software/Compiler)

- ★ Compiler can see larger scope.
 - Loop are statically unrolled by compilers.
 - Multiple instructions are statically scheduled for each cycle (see VLIW).
- ★ Poor compatibilities
- ★ Dependency cannot be determined at compile time.
- ★ Simple hardware
- ★ Lower Power requirements?

Who is the winner?



How about a chip with multicores?





Chip with multicores

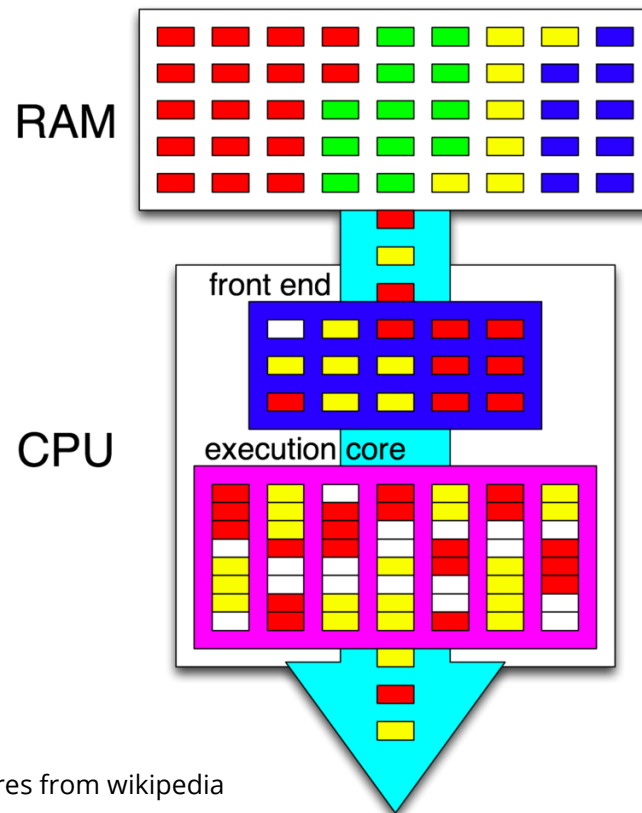
- ★ A chip with multicores is basically a chip with multiple superscalar cores inside.
- ★ Can also be viewed as Multiple Pipes.
- ★ Demands for more ILP. -- Thread-Level parallelism



Thread-Level Parallelism

The detail is beyond the scope of this course.
To learn more, join
High Performance Architecture.

- ★ Since a program (thread) cannot fill available slots in the processor, let's get instructions from several threads.
- ★ Instructions in pipeline are coming from several threads.
- ★ The technology is called simultaneous multithreading (SMT).
- ★ Intel implementation is officially called Hyper-Threading Technology.



Pictures from wikipedia



Summary

- ★ There exist several approaches to ILP.
- ★ At similar degrees, SuperScalar, SuperPipeline, VLIW will yield similar performance.
- ★ Dynamic scheduling allows legacy software to gain performance.
- ★ Static scheduling may gain more benefits if a compiler is more powerful.
 - Proven to be too difficult to have good compiler.



LLVM

- Started in 2000 at the University of Illinois @Urbana-Champaign.
- Originally developed as a research infrastructure to investigate dynamic compilation techniques for static and dynamic programming languages.
- Provide the middle layers of a complete compiler system, taking **Intermediate Representation (IR) code** from a compiler and emitting an optimized IR.

```
int mul_add(int x, int y, int z) {  
    return x * y + z;  
}
```

```
define i32 @mul_add(i32 %x, i32 %y, i32 %z) {  
entry:  
    %tmp = mul i32 %x, %y  
    %tmp2 = add i32 %tmp, %z  
    ret i32 %tmp2  
}
```




Exercises





Loop Unroll

- ★ For the given code, if MAX is a known/large value,
- (1) How many iterations can we unroll this loop?
 - (2) What is the potential speed up of the unrolled (rescheduled) code comparing to the original one.
 - (3) What will be the size of the unrolled code?
 - (4) Which resources is limited to the number of iterations that can be unrolled?

```
for (i=MAX-1;i>=0;i--) {  
    A[i] = A[i] + C;  
}
```

```
LOOP:  
    LW    $f0, $r1  
    (stall)  
    ADD   $f2, $f0,$f1  
    (stall)  
    (stall)  
    SW    $f2, $r1  
    SUB   $r1, $r1,#4  
    BNE   $r1, LOOP
```



Software Pipeline

★ For the given code, if MAX is a known/large value, please use software pipeline to optimize the code.

(1) What is the potential speed up of the unrolled (rescheduled) code comparing to the loop unroll version?

(2) What will be the size of the code?

```
for (i=MAX-1;i>=0;i--) {  
    A[i] = A[i] + C;  
}
```

```
LOOP:  
    LW    $f0, $r1  
    (stall)  
    ADD   $f2, $f0,$f1  
    (stall)  
    (stall)  
    SW    $f2, $r1  
    SUB   $r1, $r1,#4  
    BNE   $r1, LOOP
```



Dynamic SuperScalar (out-of-order execution)

★ Given a super scalar at degree two, please draw a diagram showing the pipeline of the following code.

LOOP:

```
LW    $f0, $r1
ADD    $f2, $f0,$f1
SW     $f2, $r1
SUB    $r1, $r1,#4
BNE    $r1, LOOP
```

		Issue	EX	MEM	WB	Notes
LW	\$f0, \$r1	1	2	3	4	
ADD \$f0,\$f1	\$f2,	1	5,6,7		8	Wait \$f0
SW	\$f2, \$r1	2	3	9		Wait \$f2
SUB	\$r1, \$r1,#4	2	4		5	
BNE	\$r1, LOOP	3	6			
LW	\$f0, \$r1	4	7	8	9	
ADD \$f0,\$f1	\$f2,	4	10,11,12		13	Wait \$f0
SW	\$f2, \$r1	5	8	14		
SUB	\$r1, \$r1,#4	5	9		10	
BNE	\$r1, LOOP	6	11			



End of Chapter 7

